

Name: Monisha Raghini Chelle

Roll No.: 2520030537

Section: 6

Subject: OSSP (25CS2104E)

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

Step 1 — Create

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14      Simple-Commands-Week1  foldername  hello.c  monis  sam
ple1.c
OSLAB                           OSSP-Practical  example.txt           hello       moni    sample1  sam
ple1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill        virtual_a
ddress
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
```

Step 2 — Create the Background Jobs Program

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano background_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 background_jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

#define INPUT_SIZE 200
#define MAX_JOBS 20

typedef struct
{
    int id;
    pid_t pid;
    char command[INPUT_SIZE];
    int active;
} Job;

Job jobs[MAX_JOBS];
int next_id = 1;

/* Remove leading and trailing whitespace */
char *trim(char *str)
{
    while (isspace((unsigned char)*str))
        str++;

    if (*str == '\0')
        return str;

    char *end = str + strlen(str) - 1;

    while (end > str &&
           isspace((unsigned char)*end))
    {
        *end = '\0';
        end--;
    }

    return str;
}

/* Check whether a command ends with & */
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 background_jobs.c *
/* Check whether a command ends with & */
int detect_background(char *command)
{
    char *clean = trim(command);
    size_t length = strlen(clean);

    if (length > 0 && clean[length - 1] == '&')
    {
        clean[length - 1] = '\0';
        return 1;
    }

    return 0;
}

/* Store information about a background job */
int add_job(pid_t pid, const char *command)
{
    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (!jobs[i].active)
        {
            jobs[i].id = next_id++;
            jobs[i].pid = pid;

            snprintf(
                jobs[i].command,
                sizeof(jobs[i].command),
                "%s",
                command
            );

            jobs[i].active = 1;

            return jobs[i].id;
        }
    }

    return -1;
}

/* Display currently running jobs */
void show_jobs(void)
{
    int found = 0;

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 background_jobs.c *
int found = 0;

printf("\nBackground Jobs:\n");

for (int i = 0; i < MAX_JOBS; i++)
{
    if (jobs[i].active)
    {
        printf(
            "[%d] PID=%ld RUNNING %s\n",
            jobs[i].id,
            (long)jobs[i].pid,
            jobs[i].command
        );

        found = 1;
    }
}

if (!found)
    printf("No active background jobs.\n");
}

/* Check completed background processes */
void monitor_jobs(void)
{
    int status;
    pid_t pid;

    while (1)
    {
        pid = waitpid(-1, &status, WNOHANG);

        if (pid <= 0)
            break;

        for (int i = 0; i < MAX_JOBS; i++)
        {
            if (jobs[i].active &&
                jobs[i].pid == pid)
            {
                printf(
                    "\n[%d] Background job completed. PID=%ld\n",
                    jobs[i].id,
                    (long)pid
                );
            }
        }
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 background_jobs.c *
    "\n[%d] Background job completed. PID=%ld\n",
    jobs[i].id,
    (long)pid
);

if (WIFEXITED(status))
{
    printf(
        "Exit status: %d\n",
        WEXITSTATUS(status)
    );
}
else if (WIFSIGNALED(status))
{
    printf(
        "Terminated by signal: %d\n",
        WTERMSIG(status)
    );
}

jobs[i].active = 0;
break;
}
}
}

/* Execute sleep N in a child process */
void launch_job(char *command, int background)
{
    int seconds;
    char extra;

    if (sscanf(command, "sleep %d %c",
        &seconds, &extra) != 1 ||
        seconds < 0)
    {
        printf("Error: Use sleep NUMBER.\n");
        return;
    }

    pid_t pid = fork();

    if (pid < 0)
    {

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 background_jobs.c *
if (pid < 0)
{
    perror("fork");
    return;
}

if (pid == 0)
{
    char duration[20];

    snprintf(
        duration,
        sizeof(duration),
        "%d",
        seconds
    );

    execlp(
        "sleep",
        "sleep",
        duration,
        (char *)NULL
    );

    perror("execlp");
    _exit(127);
}

if (background)
{
    int id = add_job(pid, command);

    if (id == -1)
    {
        printf("Job table full.\n");
        waitpid(pid, NULL, 0);
        return;
    }

    printf(
        "[%d] Background job started. PID=%ld\n",
        id,
        (long)pid
    );
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-F Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 background_jobs.c *
    printf("Prompt returned immediately.\n");
}
else
{
    printf("Waiting for foreground process...\n");

    if (waitpid(pid, NULL, 0) == -1)
        perror("waitpid");

    printf("Foreground process completed.\n");
}
}

int main(void)
{
    char input[INPUT_SIZE];

    printf("Background Jobs Program\n");
    printf("Supported commands:\n");
    printf("sleep 5 &\n");
    printf("sleep 2\n");
    printf("jobs\n");
    printf("check\n");
    printf("exit\n");

    while (1)
    {
        monitor_jobs();

        printf("\nbg> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

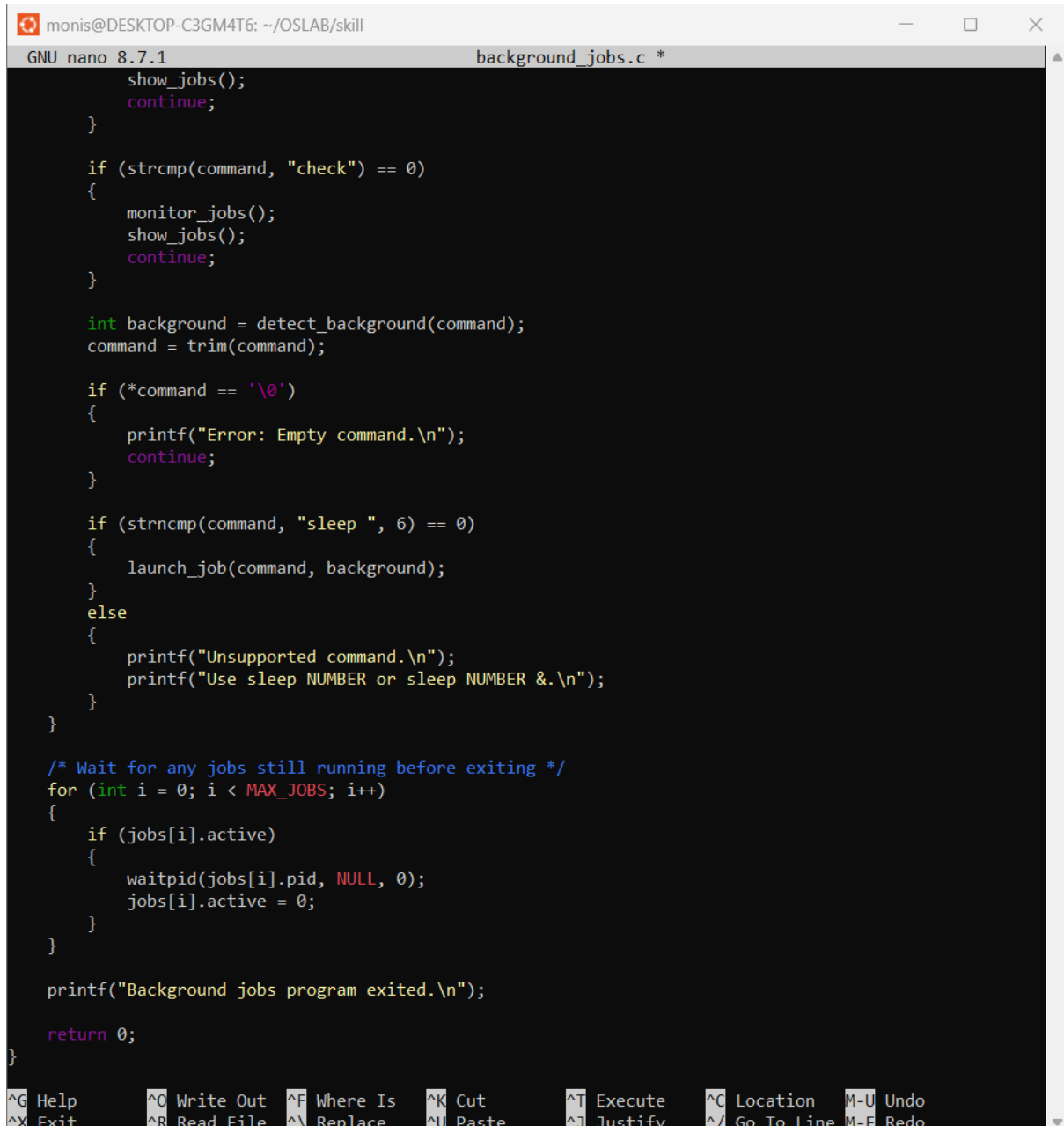
        input[strcspn(input, "\n")] = '\0';

        char *command = trim(input);

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "jobs") == 0)
        {
            monitor_jobs();
        }
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 background_jobs.c *
    show_jobs();
    continue;
}

if (strcmp(command, "check") == 0)
{
    monitor_jobs();
    show_jobs();
    continue;
}

int background = detect_background(command);
command = trim(command);

if (*command == '\0')
{
    printf("Error: Empty command.\n");
    continue;
}

if (strncmp(command, "sleep ", 6) == 0)
{
    launch_job(command, background);
}
else
{
    printf("Unsupported command.\n");
    printf("Use sleep NUMBER or sleep NUMBER &.\n");
}
}

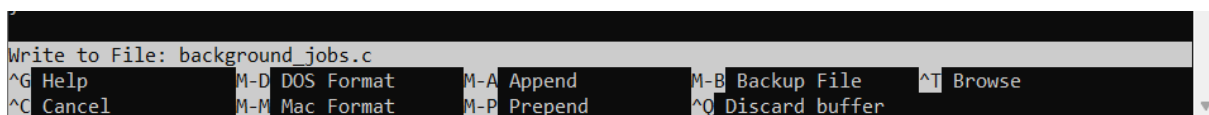
/* Wait for any jobs still running before exiting */
for (int i = 0; i < MAX_JOBS; i++)
{
    if (jobs[i].active)
    {
        waitpid(jobs[i].pid, NULL, 0);
        jobs[i].active = 0;
    }
}

printf("Background jobs program exited.\n");

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save using Ctrl + O, Enter, then Ctrl + X.



```
Write to File: background_jobs.c
^G Help M-D DOS Format M-A Append M-B Backup File ^T Browse
^C Cancel M-M Mac Format M-P Prepend ^O Discard buffer
```

Compile:

gcc -Wall -Wextra background_jobs.c -o background_jobs

Verify:

ls -l background_jobs.c background_jobs

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano background_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra background_jobs.c -o background_jobs
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l background_jobs.c background_jobs
-rwxr-xr-x 1 monis monis 17008 Sep 25 18:58 background_jobs
-rw-r--r-- 1 monis monis  5930 Sep 25 18:56 background_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 3 — Test Background Execution

Run:

./background_jobs

At the bg> prompt, enter:

sleep 15 &

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./background_jobs
Background Jobs Program
Supported commands:
sleep 5 &
sleep 2
jobs
check
exit

bg> sleep 15 &
[1] Background job started. PID=648
Prompt returned immediately.

bg>
```

Immediately enter:

jobs

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./background_jobs
Background Jobs Program
Supported commands:
sleep 5 &
sleep 2
jobs
check
exit

bg> sleep 15 &
[1] Background job started. PID=648
Prompt returned immediately.

bg> jobs

[1] Background job completed. PID=648
Exit status: 0

Background Jobs:
No active background jobs.

bg>
```

Step 4 — Monitor Background Jobs

While the program is running, wait approximately 15 seconds.

Then enter:

check

```
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano background_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra background_jobs.c -o background_jobs
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l background_jobs.c background_jobs
-rwxr-xr-x 1 monis monis 17008 Sep 25 18:58 background_jobs
-rw-r--r-- 1 monis monis 5930 Sep 25 18:56 background_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./background_jobs
Background Jobs Program
Supported commands:
sleep 5 &
sleep 2
jobs
check
exit

bg> sleep 15 &
[1] Background job started. PID=648
Prompt returned immediately.

bg> jobs

[1] Background job completed. PID=648
Exit status: 0

Background Jobs:
No active background jobs.

bg> check

Background Jobs:
No active background jobs.

bg>
```

One additional test — Capture a running job

To demonstrate the **RUNNING** state for your OSSP record, enter the following at your current **bg>** prompt:

sleep 60 &

Immediately enter:

jobs

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./background_jobs
Background Jobs Program
Supported commands:
sleep 5 &
sleep 2
jobs
check
exit

bg> sleep 15 &
[1] Background job started. PID=648
Prompt returned immediately.

bg> jobs

[1] Background job completed. PID=648
Exit status: 0

Background Jobs:
No active background jobs.

bg> check

Background Jobs:
No active background jobs.

bg> sleep 60 &
[2] Background job started. PID=677
Prompt returned immediately.

bg> jobs

Background Jobs:
[2] PID=677 RUNNING sleep 60

bg> exit
```

Step 5 — Create the Job Table Program

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano job_table.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 job_table.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 10

typedef enum
{
    EMPTY,
    RUNNING,
    COMPLETED
} JobState;

typedef struct
{
    int id;
    pid_t pid;
    pid_t pgid;
    JobState state;
    char command[100];
} Job;

Job table[MAX_JOBS];
int next_id = 1;

/* Convert a state to readable text */
const char *state_name(JobState state)
{
    switch (state)
    {
        case RUNNING:
            return "RUNNING";

        case COMPLETED:
            return "COMPLETED";

        default:
            return "EMPTY";
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 job_table.c *
int add_job(
    pid_t pid,
    pid_t pgid,
    const char *command
)
{
    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (table[i].state == EMPTY)
        {
            table[i].id = next_id++;
            table[i].pid = pid;
            table[i].pgid = pgid;
            table[i].state = RUNNING;

            snprintf(
                table[i].command,
                sizeof(table[i].command),
                "%s",
                command
            );

            return table[i].id;
        }
    }

    return -1;
}

/* Display the job table */
void display_jobs(void)
{
    int found = 0;

    printf("\nJOB TABLE\n");
    printf("ID\tPID\tPGID\tSTATE\t\tCOMMAND\n");

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (table[i].state != EMPTY)
        {
            printf(
                "%d\t%d\t%d\t%-10s\t%s\n",
                table[i].id,
                (long)table[i].pid,
            );
        }
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^J Replace   ^U Paste     ^I Justify   ^Y Go To Line ^E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 job_table.c *

        table[i].id,
        (long)table[i].pid,
        (long)table[i].pgid,
        state_name(table[i].state),
        table[i].command
    );

    found = 1;
}

if (!found)
    printf("No job records found.\n");
}

/* Update completed jobs without blocking */
void update_jobs(void)
{
    int status;

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (table[i].state != RUNNING)
            continue;

        pid_t result = waitpid(
            table[i].pid,
            &status,
            WNOHANG
        );

        if (result == table[i].pid)
        {
            table[i].state = COMPLETED;

            printf(
                "Job %d completed. PID=%ld\n",
                table[i].id,
                (long)table[i].pid
            );
        }
        else if (result == -1)
        {
            perror("waitpid");
        }
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 job_table.c *
}
}

/* Remove completed records */
void remove_completed(void)
{
    int removed = 0;

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (table[i].state == COMPLETED)
        {
            printf(
                "Removed completed job %d.\n",
                table[i].id
            );

            memset(&table[i], 0, sizeof(table[i]));
            table[i].state = EMPTY;

            removed++;
        }
    }

    if (removed == 0)
        printf("No completed jobs to remove.\n");
}

/* Validate job records */
void validate_jobs(void)
{
    int valid = 1;

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (table[i].state == EMPTY)
            continue;

        if (table[i].id <= 0 ||
            table[i].pid <= 0 ||
            table[i].pgid <= 0 ||
            table[i].command[0] == '\0')
        {
            printf(
                "Invalid record in slot %d.\n",

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 job_table.c *
    _exit(1);
}

char duration[20];

snprintf(
    duration,
    sizeof(duration),
    "%d",
    seconds
);

execlp(
    "sleep",
    "sleep",
    duration,
    (char *)NULL
);

perror("execlp");
_exit(127);
}

/*
 * The child creates its own process group.
 * Its PGID is its PID.
 */
if (setpgid(pid, pid) == -1 &&
    errno != EACCES)
{
    perror("setpgid");
}

char command[100];

snprintf(
    command,
    sizeof(command),
    "sleep %d",
    seconds
);

int id = add_job(pid, pid, command);

if (id == -1)
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^Y Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line M-F Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 job_table.c *
if (id == -1)
{
    printf("Job table is full.\n");
    waitpid(pid, NULL, 0);
    return;
}

printf(
    "Job added: ID=%d PID=%ld PGID=%ld\n",
    id,
    (long)pid,
    (long)pid
);
}

int main(void)
{
    char input[100];
    int seconds;
    char extra;

    printf("Job Table Management Program\n");

    printf("\nAvailable commands:\n");
    printf("add NUMBER\n");
    printf("list\n");
    printf("update\n");
    printf("remove\n");
    printf("validate\n");
    printf("exit\n");

    while (1)
    {
        printf("\njob> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (sscanf(input, "add %d %c",
```

Help	Write Out	Where Is	Cut	Execute	Location	Undo
Exit	Read File	Replace	Paste	Justify	Go To Line	Redo

```
    if (sscanf(input, "add %d %c",
                &seconds, &extra) == 1)
    {
        launch_job(seconds);
    }
    else if (strcmp(input, "list") == 0)
    {
        display_jobs();
    }
    else if (strcmp(input, "update") == 0)
    {
        update_jobs();
    }
    else if (strcmp(input, "remove") == 0)
    {
        update_jobs();
        remove_completed();
    }
    else if (strcmp(input, "validate") == 0)
    {
        validate_jobs();
    }
    else
    {
        printf("Invalid command.\n");
    }
}

/* Reap any remaining child processes */
for (int i = 0; i < MAX_JOBS; i++)
{
    if (table[i].state == RUNNING)
    {
        waitpid(table[i].pid, NULL, 0);
        table[i].state = COMPLETED;
    }
}

printf("Job table program exited.\n");

return 0;
}
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 job_table.c *
    printf(
        "Invalid record in slot %d.\n",
        i
    );

    valid = 0;
}

for (int j = i + 1; j < MAX_JOBS; j++)
{
    if (table[j].state != EMPTY &&
        table[i].id == table[j].id)
    {
        printf("Duplicate job ID found.\n");
        valid = 0;
    }
}

if (valid)
    printf("All job records are valid.\n");

/* Create a process group and launch a test job */
void launch_job(int seconds)

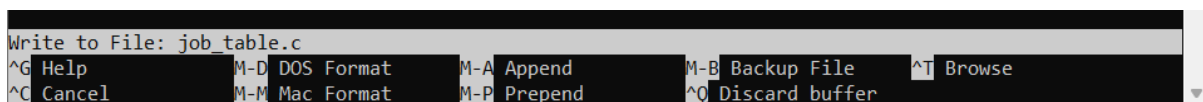
{
    if (seconds < 0)
    {
        printf("Invalid duration.\n");
        return;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        if (setpgid(0, 0) == -1)
        {
            perror("setpgid");
        }
    }
}
```

Save with Ctrl + O, Enter, then Ctrl + X.



Compile:

gcc -Wall -Wextra job_table.c -o job_table

Verify:

ls -l job_table.c job_table

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano job_table.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra job_table.c -o job_table
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l job_table.c job_table
-rwxr-xr-x 1 monis monis 17064 Sep 25 19:10 job_table
-rw-r--r-- 1 monis monis 6547 Sep 25 19:09 job_table.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 6 — Test Job Creation and Process Groups

Run:

./job_table

At the job> prompt, enter:

add 20

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./job_table
Job Table Management Program

Available commands:
add NUMBER
list
update
remove
validate
exit

job> add 20
Job added: ID=1 PID=860 PGID=860

job>
```

Immediately enter:

list

```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./job_table
Job Table Management Program

Available commands:
add NUMBER
list
update
remove
validate
exit

job> add 20
Job added: ID=1 PID=860 PGID=860

job> list

JOB TABLE
ID      PID      PGID     STATE      COMMAND
1       860      860     RUNNING    sleep 20

job>
```

Step 7 — Update Job States

After the 20-second process finishes, enter:

update

Then:

List

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./job_table
Job Table Management Program

Available commands:
add NUMBER
list
update
remove
validate
exit

job> add 20
Job added: ID=1 PID=860 PGID=860

job> list

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       860      860      RUNNING    sleep 20

job> update
Job 1 completed. PID=860

job> list

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       860      860      COMPLETED  sleep 20

job>
```

Step 8 — Validate and Remove Completed Jobs

At the same job> prompt, enter:

validate

Now enter:

remove

Check the table again:

list

Finally, enter:

exit

monis@DESKTOP-C3GM4T6: ~/OSLAB/skill

monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$./job_table

Job Table Management Program

Available commands:

add NUMBER

list

update

remove

validate

exit

job> add 20

Job added: ID=1 PID=860 PGID=860

job> list

JOB TABLE

ID	PID	PGID	STATE	COMMAND
1	860	860	RUNNING	sleep 20

job> update

Job 1 completed. PID=860

job> list

JOB TABLE

ID	PID	PGID	STATE	COMMAND
1	860	860	COMPLETED	sleep 20

job> validate

All job records are valid.

job> remove

Removed completed job 1.

job> list

JOB TABLE

ID	PID	PGID	STATE	COMMAND
No job records found.				

job> exit

Job table program exited.

monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$